Software Engineering Final Project - Mesos

1. Features & Requirements

- **Core Rules:** Complete implementation of the board game mechanics.
- **Interfaces:** Offers both a TUI (Text-based command-line) and a GUI (Graphical with real-time feedback).
- **Networking:** Supports both Socket (stream-based DTO exchange) and RMI (stubs/skeletons) protocols.
- **Leaderboard:** Persistent global ranking system using a PostgreSQL database, aggregated by player count.
- **Resilience:** Pauses 2-player matches upon disconnection (terminating after 2 minutes). Multi-player matches continue, with the disconnected player's totem left on the TurnOrderTile.

2. Division of Labor

Member	Area of Responsibility
Simone	Model (Cards, Match, Board), Network (Socket/RMI), Server/Client Controller, TUI formatting, Database execution.
Gaia	Model (Player, Tiles, CardRow), GUI implementation, Client Controller architecture.
Giacomo	Model (Turn Logic, GameManager), Network logic, Server/Client Controller, TUI logic, Player Persistence.

3. Implementation Choices and Design Patterns Used

3.1 Design Patterns Used

- **MVC:** The architecture is structured around the model-view-control pattern, which separates the internal game logic (`Model`) from the user interface representations (`View`) and the coordination logic (`Controller`)
- **Factory:** Used to centralize card instantiation. This approach enforces structural consistency while delegating the specific implementation of card effects to concrete subclasses
- **Observer:** Used to push game-state updates to every client without the controller knowing the underlying transport. The `Server` keeps a registry (`Map<String, VirtualView>`) of the connected clients' `VirtualViews`, which act as the observers; a client subscribes on `connect` through `registerRmi`/`registerSocket`. Whenever the game state changes, the `ServerController` notifies all subscribers via its `broadcast`/`broadcastExcept` helpers, which iterate over the registry and invoke the matching `onXxx` event method on each `VirtualView`. Since RMI stubs and the `Socket_VirtualClient` are interchangeable `VirtualView` implementations, the notification logic stays protocol-agnostic.
- **Visitor:** Used to decouple the socket message transport logic from the game business logic by defining a `Visitor` interface with a `visitXxx` method for every specific message type. On the server, `Socket_ClientHandler` implements this interface, allowing it to receive a polymorphic `ClientToServerMessage`, call its `accept(this)` method, and automatically dispatch execution to the correct handler method in the `ServerController`. The same logic applies on the client, where `Socket_Client` implements the `ServerToClientMessage.Visitor` interface to route incoming network packets to the `ClientController`
- **State Pattern:** implemented server-side in the `controller.server` package, where the current game phase is encapsulated into distinct `PhaseLogic` classes. The `ServerController` holds a `PhaseLogic` field and changes its behavior dynamically.
- **Singleton:** Used in `DatabaseManager`: It's used to centralize database resource management and prevent thread-safety issues caused by multiple, uncoordinated connections by providing a single, controllable access point for infrastructure operations.

3.2 Architectural Choices

- **TUI:** The implementation is split between two main components: `TuiPrinter` is a utility class whose purpose is to output formatting, it doesn't perform any I/O operation, `TuiView` handles the input/output flow, it interacts directly with the user via console.
- **VirtualServer/VirtualView:** The interfaces act as a communication abstraction layer, decoupling the game logic from the underlying network protocols. By defining a unified contract for both RMI and Socket implementations, it ensures that the protocols are fully interchangeable.
- **GUI:** The implementation is split into two main logical areas: the FXML Controllers are responsible for updating the visual components and listening for direct user inputs, while `GuiView` acts as the central mediator that implements the core View interface, coordinating the windows and managing the asynchronous communication between the background network thread and the UI.

4. Communication Protocols

Clients choose their preferred protocol at startup. The server runs both transports simultaneously, tracking all clients via a centralized registry. Game updates are pushed out contextually: directly to Socket streams, and via single-thread executors for RMI clients to prevent race conditions and preserve message order.

- **RMI:** Relies on remote stubs. Clients invoke server methods which forward requests to the `ServerController`. The server pushes updates directly to the client's registered local stub.
- **Socket:** Message-based architecture utilizing the Visitor pattern. Actions are serialized as DTOs. Daemon threads constantly read incoming data and route it automatically, meaning the application logic never handles raw TCP streams.

5. Test Coverage

Model (it.polimi.ingsw.model) Coverage: 100%

The project is verified through exhaustive unit and integration testing. The testing suite focuses on handling core game mechanics, complex edge cases, and accurate state transitions.

Element	Class, %	Method, %	Line, %	Branch, %
database	0% (0/2)	0% (0/10)	0% (0/73)	0% (0/20)
exceptions	100% (2/2)	50% (2/4)	50% (2/4)	100% (0/0)
model	100% (52/52)	86% (243/2...	91% (745/81...	86% (288/3...
card	100% (38/38)	77% (104/13...	87% (208/2...	100% (80/80)
enums	100% (4/4)	100% (8/8)	100% (29/29)	100% (0/0)
Board	100% (1/1)	91% (22/24)	85% (110/128)	73% (60/82)
CardRow	100% (1/1)	94% (16/17)	95% (62/65)	87% (28/32)
Decks	100% (2/2)	100% (6/6)	98% (103/10...	85% (34/40)
GameManager	100% (1/1)	100% (3/3)	100% (13/13)	100% (8/8)
Match	100% (1/1)	95% (19/20)	90% (85/94)	73% (34/46)
OfferingTile	100% (1/1)	100% (11/11)	100% (17/17)	100% (2/2)
Player	100% (1/1)	95% (38/40)	97% (73/75)	100% (14/14)
TurnOrderSlot	100% (1/1)	100% (7/7)	100% (13/13)	100% (4/4)
TurnOrderTile	100% (1/1)	100% (9/9)	100% (32/32)	100% (24/24)